Artificial Intelligence Assignment I

基於深度學習之瑜珈姿勢辨識

姓名：陳瑤光 學號：410551015

GitHub:https://github.com/YaoKuang/AI/tree/main/Hw1

一、研究問題與動機

瑜珈姿勢本質上與人體骨架結構、關節角度與身體幾何配置高度相關，因此非常適合作為姿態特徵導向分類問題的研究題目。相較於直接將原始影像輸入深度模型，姿態特徵具有更好的可解釋性，能更清楚分析模型究竟是根據哪些身體結構資訊完成判斷。本作業強調資料集建構與分析，而瑜珈影像可透過網路蒐集與標註，自行建立一個具明確類別定義、規模適中且可進行多種分析實驗的資料集，因此相當符合本作業的要求與精神。

二、方法

1. 方法概述

本研究將瑜伽姿勢辨識視為一個多類別分類問題。與直接使用原始影像像素訓練分類器不同，本研究先透過 MediaPipe Pose 將每張影像轉換為結構化的人體姿態表示，再將這些姿態特徵作為傳統監督式學習模型的輸入。此設計可提升方法的可解釋性，並降低背景、顏色與光線等非姿勢因素對分類結果的影響。

1.1 MediaPipe

近年來，愈來愈多的組織開發了各種的身體動作或姿態檢測方法，例如 2014 由史丹弗大學提出的 DeepPose、2015 年由 Joseph Redmon 提的出 YOLO 、2016 年由馬克斯·普朗克研究所的 OpenPose 和 2020 年由 Google 提出的 BlazePose，

在本論文中，我們採用了 Google 開發的 MediaPipe 框架，該框架中的姿勢估計功能主要基於 BlazePose 模型來實現。以下是關於不同姿勢估計模型的比較：

表 1、姿勢估計模型比較

架構	特色	優點	缺點	關鍵點數量
BlazePose	使用基於深度學習的系統，透過一系列輕量的神經網路來從健身運動的影片中提取特徵。	快速、高度優化，可以在各種邊緣設備上運行。	需要大量已標記的數據進行訓練。	33 個
DeepPose	使用深度卷積神經網路從健身運動的影片中提取特徵。	準確、高效。	需要大量已標記的數據進行訓練。	14 個
OpenPose	使用一個實時多人關鍵點檢測庫進行身體、臉部、手部和腳部的估計。	可以在單個模型中估計多人的完整姿態。	需要較大的計算能力。	18 個或 25 個
YOLO	將影像切分為多個格子，並在每個格子中使用單一神經網路預測影像中物體的定界框和類別標籤。	極高的速度，適合於實時應用。	在檢測小物體方面表現不夠好。	17 個

2. 姿態特徵擷取

對每張輸入影像，首先使用 MediaPipe Pose Landmarker 偵測人體骨架關鍵點。若某張影像無法成功偵測出人體姿態，則將其排除於最終特徵資料集之外。對於成功處理的影像，則將人體姿態轉換為數值特徵並存入 CSV 檔，以供後續分類模型使用。

本研究萃取的特徵主要分為三類。

2.1 正規化骨架關鍵點特徵

本研究先對偵測到的人體 landmark 進行正規化。具體而言，以左右髖部中點作為身體中心，以肩膀中點與髖部中點之間的距離作為 torso size。每個 landmark 的 (x, y, z)座標皆以此中心與尺度進行正規化，以減少人物在影像中

的絕對位置與身形尺度差異造成的影響。此外，每個 landmark 的 visibility 值也一併納入特徵中。

2.2 關節角度特徵

為了更明確描述人體姿勢，本研究根據 2D landmark 座標計算多個關節角度，包括左右手肘、左右肩膀、左右髖部與左右膝蓋角度。這些角度特徵能夠反映肢體局部配置，對區分不同瑜伽姿勢具有重要作用。

2.3 幾何與比例特徵

除了 landmark 與角度外，本研究也計算了多個幾何特徵，包括肩寬、髖寬、軀幹長度、肩髖比例、左右手臂比例與左右腿部比例等。這些特徵可進一步補充人體姿勢的結構資訊，使分類器更容易捕捉姿勢之間的差異。

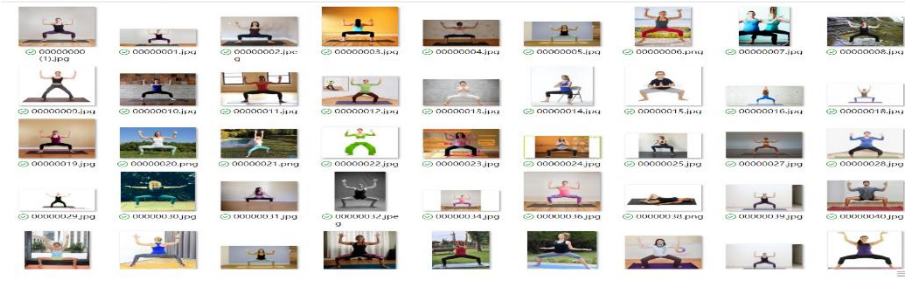
3. 數據收集與處理

本研究資料主要透過網路公開圖片蒐集而得，並以自動化或半自動化方式整理。資料來源為公開可取得之網頁圖片資源，研究中僅將其用於課堂作業之資料集建構與模型分析，不作商業用途。

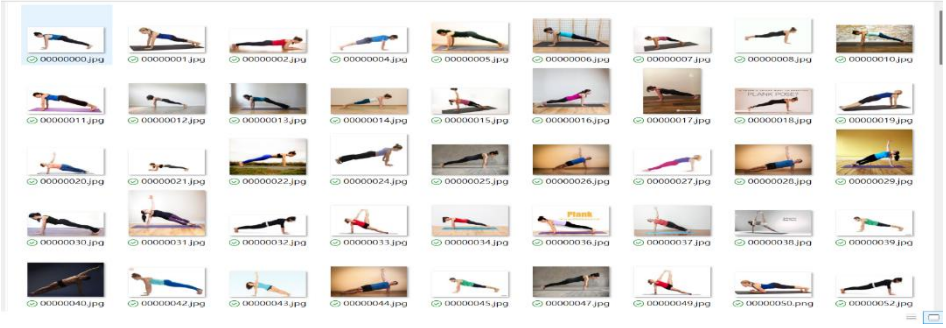
將瑜伽姿勢辨識設定為一個五類別監督式分類問題，目標類別為 downdog、goddess、plank、tree、warrior2。與直接將原始影像輸入分類器不同，本研究先利用 MediaPipe Pose 將每張影像轉換為人體姿態表示，再進一步萃取出多種數值特徵，最後以傳統機器學習模型進行分類。這樣的設計可讓分類器聚焦於人體姿勢本身，而不是受到背景、光線、服裝或影像外觀差異的過度影響，同時也讓整個系統更具可解釋性。

(1)Downdog

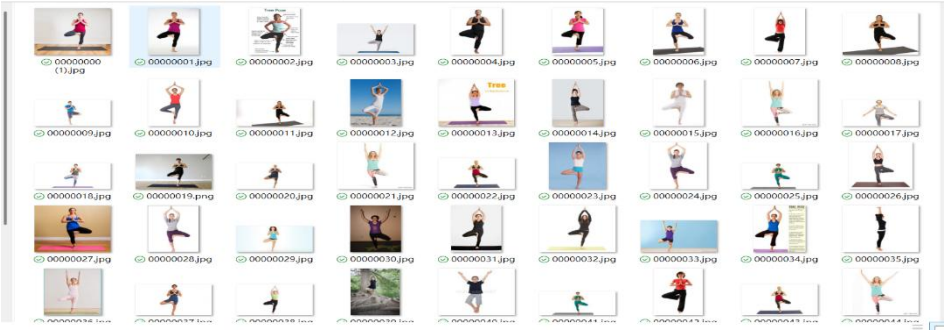
(2)Goddess



(3)Plank



(4)Tree



(5)warrior2

在特徵擷取階段，程式會先對每張影像使用 MediaPipe Pose Landmarker 進行人體骨架偵測。若某張影像無法成功偵測出骨架，則該影像不納入後續分析；若偵測成功，便將偵測結果轉換為表格型數值特徵。這些特徵主要包含三個部分。第一部分是正規化後的人體 landmark 座標與 visibility 值。研究中以左右髖部中點作為人體中心，以肩膀中點到髖部中點的距離作為 torso size，進行座標正規化，藉此降低人物在畫面中絕對位置與身體大小不同所造成的影響。第二部分是關節角度特徵，包括左右手肘、左右肩膀、左右髖部與左右膝蓋角度，這些特徵可直接反映肢體的彎曲與伸展狀態。第三部分則是幾何與比

例特徵，例如肩寬、髖寬、軀幹長、肩髖比例、手臂比例與腿部比例等，用以補充整體身體結構資訊。

為了使不同類別之間的姿勢特徵較具可比性，本研究在資料蒐集時設定以下條件：

(1)影像中需以單一人體為主。

(2)盡量保留全身或大部分關鍵肢體可見的影像。

(3)姿勢需能清楚辨認為五個目標類別之一。

(4)排除過度模糊、遮擋嚴重、人體比例過小或非主要主體的影像。

(5)若姿勢明顯不符類別定義，則不納入資料集。

透過上述條件，可降低標註模糊與骨架偵測失敗的情況，提高後續分析品質。

本研究資料蒐集與處理流程使用以下工具：

(1)Python

(2)OpenCV：讀取與處理影像

(3)MediaPipe Pose Landmarker：人體骨架偵測

(4)NumPy / Pandas：數值與表格處理

(5)scikit-learn：模型訓練與評估

若以實作角度來看，MediaPipe 是本研究中最關鍵的工具，因為它將原始影像轉換為可解釋的人體姿態表示，使後續分類器能夠聚焦於姿勢本身而非背景外觀。

本研究原始資料集由五個瑜珈姿勢類別組成，每一類皆蒐集一定數量的圖片。

downdog：216 張、goddess：200 張、plank：235 張、tree：189 張、warrior2：166 張、原始總數：1006 張

經過 MediaPipe 骨架偵測與清理後，成功保留為有效樣本的資料如下：

有效樣本總數：976 張

骨架偵測失敗或清理排除張數：29 張

```
Saved features to: c:\Users\purpl\OneDrive\Documents\人工智慧\HW1\pose_features.csv
Valid images: 976
Failed images: 29
```

在最終建模階段，本研究採用分層抽樣進行資料切分，以維持各類別比例大致一致。切分方式如下：70% training set、10% validation set、20% test set，由於本研究資料包含網路蒐集圖片，因此資料清理是必要步驟。主要清理方式如下：

(1)骨架偵測失敗排除：若 MediaPipe 無法成功偵測人體姿態，則該影像不納入最終特徵資料。
(2)不符合類別定義排除：若圖片中的姿勢與目標類別差異過大，則不保留。
(3)低品質影像排除：包含嚴重模糊、遮擋過多、人體主體過小或多人干擾明顯的影像。
(4)格式整理：統一將成功樣本轉換為表格型特徵資料，以便進行後續模型訓練。

這些清理步驟可降低資料噪音，並提升特徵品質與模型穩定性。

4. 資料表示方式

完成特徵擷取後，每張影像在 pose_features.csv 中對應為一列資料。資料中的 filename、filepath 與 label 等欄位不作為模型輸入，其餘數值欄位則作為特徵向量。類別標籤之後再以 label encoding 轉換成數值形式，以便模型訓練。

5. 資料切分策略

為了公平評估模型表現，本研究以分層抽樣方式將資料切分為 training、validation 與 test 三部分，使各子集合中的類別分布盡量與原始資料一致。比例分別為：70% training、10% validation、20% test ，其中，training set 用於模型訓練，validation set 用於模型比較與選擇，test set 則保留作為最終泛化能力評估。此外，為符合作業要求，本研究進一步在 training set 上進行 5-fold stratified cross-validation。此作法可提供較穩定的模型估計結果，避免只依賴單一次資料切分而造成偏差。

6. 分類模型

本研究比較兩種監督式學習模型。

6.1 支援向量機（SVM）

第一個模型為使用 RBF kernel 的支援向量機。在分類前，先使用平均值填補法處理遺漏值，再使用 StandardScaler 進行特徵標準化。由於本研究特徵同時包含座標、角度、距離與比例，不同特徵尺度不一致，因此標準化對 SVM 尤其重要。SVM 的超參數設定為 $C = 1.0$、gamma = scale。

6.2 隨機森林（Random Forest）

第二個模型為 Random Forest 分類器。和 SVM 一樣，先以平均值填補遺漏值。Random Forest 設定使用 300 棵樹，並固定 random seed 以確保結果可重現。由於 Random Forest 本身對特徵尺度較不敏感，故不需額外進行標準化。

7. 評估指標

本研究使用多種分類評估指標，包括：accuracy 、macro-F1 、confusion matrix 、classification report 、Accuracy 用於衡量整體分類正確率；macro-F1 則對每個類別給予相同權重，因此在多類別問題中能更全面反映模型表現。Confusion matrix 可用來觀察哪些姿勢類別之間較容易混淆，而 classification report 則提供更細部的 precision、recall 與 F1-score。這些指標皆在 validation set 與 test set 上進行評估。

整體結果顯示，無論是 SVM 或 RandomForest，兩者都能在這組姿態特徵上達到很高的分類效果，表示本研究所萃取的 landmarks、關節角度與身體比例特徵，確實已經足以區分五種瑜伽姿勢。不過，RandomForest 的整體表現明顯優於 SVM。

在 validation set（196 筆）上，RandomForest 的 accuracy 為 0.9592，macro-F1 為 0.9584；而 SVM 的 accuracy 為 0.9439，macro-F1 為 0.9416。這表示在模型比較階段，RandomForest 對這組特徵的掌握能力較佳。

在 test set（98 筆）上，RandomForest 的表現進一步提升到 accuracy 0.9796、macro-F1 0.9770，而 SVM 的結果則為 accuracy 0.9490、macro-F1 0.9467。由於兩個模型在 test set 上仍維持高表現，顯示本研究方法具備不

錯的泛化能力；而 RandomForest 在最終評估中仍保持優勢，進一步證明其更適合目前這組表格型姿態特徵。

從類別別結果來看，RandomForest 在 validation set 上的各類別 F1-score 分別為：downdog 0.9744、goddess 0.9351、plank 0.9691、tree 0.9610、warrior2 0.9524。這表示 RandomForest 對五個類別都維持相當均衡的辨識能力，沒有明顯失敗的類別。其中略低的是 goddess，但整體仍屬高表現。到了 test set，RandomForest 在 downdog、plank、tree 三類甚至達到 1.0000 的 F1-score，而 goddess 與 warrior2 也分別達到 0.9474 與 0.9375，可見其最終結果相當穩定。

SVM 雖然略遜於 RandomForest，但仍有相當不錯的表現。在 validation set 上，SVM 的各類別 F1-score 分別為：downdog 0.9620、goddess 0.9211、plank 0.9691、tree 0.9351、warrior2 0.9206；在 test set 上則為 downdog 0.9744、goddess 0.9231、plank 0.9583、tree 0.9744、warrior2 0.9032。可以看出 SVM 對 plank 與 downdog 的辨識也相當強，但在 goddess、tree、warrior2 這幾個站姿類別上較容易受到影響，尤其 warrior2 是 SVM 最困難的類別。

從姿勢特性來看，plank 應該是最容易辨識的類別之一，因為它具有明顯的近水平身體配置，與其他姿勢差異很大；downdog 也同樣容易辨識，因其倒 V 形的整體結構具有很高的辨識性。相較之下，goddess、tree、warrior2 都屬於相對直立的姿勢，可能在軀幹方向或局部肢體安排上存在部分相似性，因此比較容易產生混淆。這也與模型結果中這些類別分數略低的現象一致。

另一個值得注意的現象是，兩個模型在 test set 上的表現都略高於 validation set。以 RandomForest 為例，validation accuracy 為 0.9592，而 test accuracy 為 0.9796；SVM 則從 validation accuracy 0.9439 提升到 test accuracy 0.9490。這通常表示本次切分下的 test subset 可能剛好略容易，而不一定代表模型過擬合。由於兩模型在 validation 與 test 的類別表現趨勢一致，因此整體結論仍具可信度。

8. 基本模型比較實驗

本研究的核心實驗是比較兩種監督式學習模型：SVM 與 RandomForest。兩者皆使用完全相同的姿態特徵資料集進行訓練與評估，因此可以公平比較模型本身在相同特徵空間中的行為差異。本研究使用的特徵包括正規化 landmark、關節角度與人體幾何比例特徵，整體形成一組結構化表格型資料。

資料切分方式為 70% training、10% validation、20% test，並且在 training set 上進行 5-fold stratified cross-validation。這樣的設計可以從三個層面分析模型：第一是 training set 上 cross-validation 的穩定性；第二是 validation set 上的模型比較；第三是 held-out test set 上的最終泛化能力。

此實驗的目的，是驗證以姿態為基礎的特徵表示是否足以區分五種瑜伽姿勢，並比較 kernel-based 模型與 tree-based 模型何者較適合目前這組特徵空間。

9. Validation 與 Test 評估實驗

本研究另一個重要實驗設計，是同時使用 **validation set** 與 **held-out test set** 進行評估，而不是只依賴單一 train/test 切分。Validation set 用於模型比較與中間分析，test set 則保留作為最終結果報告。這樣的設計可以減少只看單一資料切分而造成的偶然性。

結果顯示，在 validation set 上，RandomForest 的 **accuracy 為 0.9592，macro-F1 為 0.9584**，而 SVM 的 **accuracy 為 0.9439，**macro-F1 為 0.9416。在最終 test set 上，RandomForest 的表現進一步達到 accuracy 0.9796、macro-F1 0.9770，SVM 則為 accuracy 0.9490、macro-F1 0.9467。由於兩模型在 test set 上仍維持高表現，表示這組姿態特徵對未見資料也具有不錯的泛化能力；而 RandomForest 一直優於 SVM，則暗示目前特徵中可能存在較多非線性交互關係，更適合由樹模型捕捉。

三、討論

1.類別分析實驗

除了整體 accuracy 之外，本研究也透過 confusion matrix 與 classification report，分析 **哪些類別較容易辨識、哪些類別較容易混淆**。這個實驗很重要，因為不同瑜伽姿勢在結構上可能有不同程度的區辨性。有些姿勢具有非常明顯的整體形狀，而有些姿勢之間則可能在局部肢體排列上具有相似性。

結果顯示，**plank** 與 **downdog** 是最容易辨識的類別之一。以 RandomForest 為例，plank 在 validation set 的 F1-score 為 0.9691，在 test set 則達到 1.0000；downdog 在 validation set 為 0.9744，在 test set 也達到 1.0000。這可能是因為 plank 具有明顯的近水平軀幹排列，而 downdog 則具有非常典型的倒 V 型結構，因此在幾何特徵空間中較容易與其他姿勢分開。相較之下，goddess、tree、warrior2 這幾個站姿類別在某些情況下表現略低，特別是 SVM。以 SVM 的 test set 為例，warrior2 的 F1-score 僅 0.9032，是五個類別中最低者。這表示 warrior2 可能和其他站姿類別在 landmark、角度或局部幾何結構上存在部分重疊，因此較不容易完全分開。這個類別別分析實驗說明，雖然整體特徵表示效果已經很好，但不同姿勢本身的結構差異仍會影響分類難度，而這些現象也可作為後續 discussion 的重要依據。

2. 模型行為比較實驗

本研究也進一步分析兩個模型在相同特徵表示下的行為差異。結果很明顯地顯示 RandomForest 在 validation 與 test 上都優於 SVM，而且這種差距不只反映在整體 accuracy，也體現在類別別 F1-score 上。例如在 validation set 中，RandomForest 在 goddess、tree 與 warrior2 上都優於 SVM；在 test set 上也維持類似趨勢。

這表示目前的姿態特徵空間，可能不是單純用一個平滑的 kernel decision boundary 就能完全描述，而是存在較多特徵間的非線性交互作用；這種特性正好較適合由 RandomForest 這類樹狀集成模型來處理。因此，模型比較實驗不只是找出較好的分類器，也進一步幫助理解目前特徵表示的結構特性。

四、 實驗總結

整體而言，本研究的實驗支持三個主要結論。第一，以人體姿態為基礎的特徵表示，已足以有效完成瑜伽姿勢分類，即使不使用端到端深度影像模型也能取得高品質結果。第二，在目前這組表格型姿態特徵上，RandomForest 明顯比 SVM 更適合。第三，類別別分析顯示 plank 與 downdog 因結構明顯而較容易辨識，而 warrior2、goddess 等站姿則相對較具挑戰性。這些實驗說明，本專題不只是得到一個高 accuracy 數字而已，而是進一步理解資料集特性、特徵設計與模型選擇之間的互動關係。

五、 附件

1. 程式

(1.) final extract_pose_features.py 程式用途:
逐類別讀取瑜伽圖片，用 MediaPipe 偵測人體骨架，把骨架轉成可用於機器學習的數值特徵
特徵包含：
正規化 landmarks
關節角度
人體幾何比例
最後輸出成 pose_features.csv，給後面的 SVM / RandomForest 使用。

```
# 匯入作業系統相關功能，用來處理路徑、資料夾、檔案是否存在等操作
import os

# 匯入 OpenCV，主要用來讀圖片、轉換色彩空間
import cv2

# 匯入 NumPy，主要用來做向量、矩陣、距離、角度等數值運算
import numpy as np

# 匯入 Pandas，主要用來把特徵整理成 DataFrame 並輸出成 CSV
import pandas as pd

# 匯入 MediaPipe 主套件，用來做人體姿態偵測
import mediapipe as mp

# 從 mediapipe.tasks 匯入 python API 基底設定
from mediapipe.tasks import python

# 匯入 vision 模組，裡面有 PoseLandmarker 相關類別
from mediapipe.tasks.python import vision


# ===== 路徑設定 =====

# 取得目前這支 Python 檔案所在資料夾的絕對路徑
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
# 設定資料集圖片資料夾位置：目前程式預期圖片放在
BASE_DIR/dataset/image
DATASET_DIR = os.path.join(BASE_DIR, "dataset", "image")

# 設定輸出的特徵 CSV 檔案名稱與路徑
OUTPUT_CSV = os.path.join(BASE_DIR, "pose_features.csv")

# 允許讀取的圖片副檔名
IMG_EXTENSIONS = (".jpg", ".jpeg", ".png")


# ===== 類別名稱 =====

# 這裡列出你的五個瑜伽姿勢類別
# 注意：資料夾名稱必須和這裡完全一致，否則程式找不到對應類別資料夾
CLASSES = ["tree", "warrior2", "goddess", "plank", "downdog"]


# ===== MediaPipe Pose Landmarker 模型 =====

# 設定 MediaPipe 模型要放在哪個資料夾
MODEL_DIR = r"C:\temp\mediapipe_models"

# 設定 pose landmarker 模型檔完整路徑
MODEL_PATH = os.path.join(MODEL_DIR, "pose_landmarker_lite.task")

# 設定官方模型下載網址
MODEL_URL = "https://storage.googleapis.com/mediapipe-
models/pose_landmarker/pose_landmarker_lite/float16/latest/pose_landmarker_lite.ta
sk"


# ===== 常用 landmark index =====

# 以下是 MediaPipe 人體關鍵點索引
# 之後計算角度、距離、比例時會直接用這些名稱，讓程式可讀性更高
```

```
# 鼻子 landmark 的 index
NOSE = 0

# 左肩膀 landmark 的 index
LEFT_SHOULDER = 11

# 右肩膀 landmark 的 index
RIGHT_SHOULDER = 12

# 左手肘 landmark 的 index
LEFT_ELBOW = 13

# 右手肘 landmark 的 index
RIGHT_ELBOW = 14

# 左手腕 landmark 的 index
LEFT_WRIST = 15

# 右手腕 landmark 的 index
RIGHT_WRIST = 16

# 左髖部 landmark 的 index
LEFT_HIP = 23

# 右髖部 landmark 的 index
RIGHT_HIP = 24

# 左膝蓋 landmark 的 index
LEFT_KNEE = 25

# 右膝蓋 landmark 的 index
RIGHT_KNEE = 26

# 左腳踝 landmark 的 index
LEFT_ANKLE = 27

# 右腳踝 landmark 的 index
RIGHT_ANKLE = 28
```

```
# 定義函式：確保 MediaPipe 模型存在
def ensure_model_exists():
    # 若模型資料夾不存在，就先建立
    os.makedirs(MODEL_DIR, exist_ok=True)

    # 如果模型檔不存在，或存在但檔案大小為 0，則重新下載
    if (not os.path.exists(MODEL_PATH)) or os.path.getsize(MODEL_PATH) == 0:
        # 印出提示訊息，表示要下載模型
        print(f"[Info] Downloading model to: {MODEL_PATH}")

        # 在函式內匯入 urllib.request，用來下載檔案
        import urllib.request

        # 從官方網址下載模型檔到本機指定路徑
        urllib.request.urlretrieve(MODEL_URL, MODEL_PATH)

        # 印出下載完成訊息
        print("[Info] Model downloaded successfully.")

    # 印出模型完整路徑
    print("MODEL_PATH =", MODEL_PATH)

    # 印出模型是否存在
    print("Model exists =", os.path.exists(MODEL_PATH))

    # 若模型存在，再印出模型檔大小
    if os.path.exists(MODEL_PATH):
        print("Model size =", os.path.getsize(MODEL_PATH), "bytes")


# 定義函式：建立 PoseLandmarker 物件
def create_pose_landmarker():
    # 設定基本選項：指定模型檔路徑
    base_options = python.BaseOptions(model_asset_path=MODEL_PATH)

    # 設定 PoseLandmarker 的各項參數
```

```
    options = vision.PoseLandmarkerOptions(
        # 傳入基本模型設定
        base_options=base_options,

        # 指定執行模式為單張圖片模式
        running_mode=vision.RunningMode.IMAGE,

        # 每張圖最多只抓 1 個人體
        num_poses=1,

        # 人體偵測最低信心分數門檻
        min_pose_detection_confidence=0.5,

        # 人體存在的最低信心分數門檻
        min_pose_presence_confidence=0.5,

        # 追蹤最低信心分數門檻
        min_tracking_confidence=0.5,
    )

    # 根據設定建立並回傳 PoseLandmarker
    return vision.PoseLandmarker.create_from_options(options)


# 定義函式：安全讀取含中文路徑的圖片
def read_image_unicode(img_path):
    # 如果檔案路徑不存在，印錯誤並回傳 None
    if not os.path.exists(img_path):
        print("[Path not found]", img_path)
        return None

    # 先用 np.fromfile 讀二進位資料，這種做法比較能處理中文路徑
    data = np.fromfile(img_path, dtype=np.uint8)

    # 若讀到的資料長度是 0，表示檔案空白或不可讀
    if data.size == 0:
        print("[Empty file or unreadable]", img_path)
        return None
```

```
    # 使用 cv2.imdecode 將二進位資料解碼成圖片
    img = cv2.imdecode(data, cv2.IMREAD_COLOR)

    # 若解碼失敗，印出訊息
    if img is None:
        print("[cv2.imdecode failed]", img_path)

    # 回傳讀到的圖片
    return img


# 定義函式：計算兩點之間的歐式距離
def calculate_distance(a, b):
    # 將 a、b 轉成 NumPy 陣列後相減，再用 norm 計算距離
    return np.linalg.norm(np.array(a) - np.array(b))


# 定義函式：計算三點形成的角度 ABC
def calculate_angle(a, b, c):
    """
    angle ABC, with b as the vertex
    a, b, c are (x, y)
    """

    # 將三個點都轉成 NumPy 陣列，方便做向量運算
    a = np.array(a)
    b = np.array(b)
    c = np.array(c)

    # 建立向量 BA：從 b 指向 a
    ba = a - b

    # 建立向量 BC：從 b 指向 c
    bc = c - b

    # 分母 = |BA| * |BC|，用來算 cosine
    denom = np.linalg.norm(ba) * np.linalg.norm(bc)
```

```
    # 若分母為 0，表示其中一條向量長度為 0，避免除以 0
    if denom == 0:
        return 0.0

    # 根據內積公式計算 cos(theta)
    cos_angle = np.dot(ba, bc) / denom

    # 避免浮點誤差導致數值超出 [-1, 1]
    cos_angle = np.clip(cos_angle, -1.0, 1.0)

    # 使用 arccos 取得弧度，再轉成角度
    angle = np.degrees(np.arccos(cos_angle))

    # 回傳角度值
    return angle


# 定義函式：對人體 landmarks 做正規化
def normalize_landmarks(landmarks):
    """
    用骨架中心 + 身體尺度做正規化
    中心點：左右髖中點
    尺度：左右肩中點到左右髖中點的 torso length
    """

    # 取左髖的 (x, y)
    left_hip = np.array([landmarks[LEFT_HIP].x, landmarks[LEFT_HIP].y])

    # 取右髖的 (x, y)
    right_hip = np.array([landmarks[RIGHT_HIP].x, landmarks[RIGHT_HIP].y])

    # 左右髖中點作為身體中心
    hip_center = (left_hip + right_hip) / 2.0

    # 取左肩的 (x, y)
    left_shoulder = np.array([landmarks[LEFT_SHOULDER].x,
landmarks[LEFT_SHOULDER].y])
```

```
    # 取右肩的 (x, y)
    right_shoulder = np.array([landmarks[RIGHT_SHOULDER].x,
landmarks[RIGHT_SHOULDER].y])

    # 左右肩中點
    shoulder_center = (left_shoulder + right_shoulder) / 2.0

    # 以肩中點到髖中點的距離作為 torso 尺度
    torso_size = np.linalg.norm(shoulder_center - hip_center)

    # 若 torso 太小，避免後面除以 0，直接設成 1
    if torso_size < 1e-6:
        torso_size = 1.0

    # 建立空串列，之後存每個 landmark 的正規化結果
    norm_points = []

    # 逐一處理每個 landmark
    for lm in landmarks:
        # x 座標減去 hip_center，再除以 torso_size
        x = (lm.x - hip_center[0]) / torso_size

        # y 座標減去 hip_center，再除以 torso_size
        y = (lm.y - hip_center[1]) / torso_size

        # z 座標直接除以 torso_size
        z = lm.z / torso_size

        # 若 landmark 有 visibility 屬性就取它，沒有就預設 1.0
        vis = getattr(lm, "visibility", 1.0)

        # 把正規化後的 (x, y, z, vis) 存進串列
        norm_points.append((x, y, z, vis))

    # 回傳所有正規化後的 landmark
    return norm_points
```

```
# 定義函式：從單張圖片擷取姿勢特徵
def extract_pose_features(image_path, landmarker):
    # 讀取圖片
    image = read_image_unicode(image_path)

    # 若讀圖失敗，印出訊息並回傳 None
    if image is None:
        print("[Read failed]", image_path)
        return None

    # 印出圖片 shape 與路徑，方便除錯
    print("shape =", image.shape, "path =", image_path)

    # OpenCV 讀到的是 BGR，先轉成 RGB
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # 將 NumPy 圖片包成 MediaPipe Image 物件
    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB,
data=image_rgb)

    # 執行姿態偵測
    result = landmarker.detect(mp_image)

    # 若沒有偵測到人體骨架，印出訊息並回傳 None
    if not result.pose_landmarks:
        print("[No pose detected]", image_path)
        return None

    # 只取第一個人
    landmarks = result.pose_landmarks[0]

    # 將 landmarks 做正規化
    norm_points = normalize_landmarks(landmarks)

    # 建立空字典，之後把所有特徵放進去
    features = {}
```

```
# ===== 1. normalized landmarks =====
# 把每個 landmark 的 x, y, z, visibility 都存成獨立欄位
for i, (x, y, z, v) in enumerate(norm_points):
    # 第 i 個 landmark 的 x
    features[f"lm_{i}_x"] = x

    # 第 i 個 landmark 的 y
    features[f"lm_{i}_y"] = y

    # 第 i 個 landmark 的 z
    features[f"lm_{i}_z"] = z

    # 第 i 個 landmark 的 visibility
    features[f"lm_{i}_vis"] = v

# 只抓 2D 點來計算角度
pts = [(landmarks[i].x, landmarks[i].y) for i in range(len(landmarks))]

# ===== 2. joint angles =====

# 左手肘角度：肩膀-手肘-手腕
features["angle_left_elbow"] = calculate_angle(
    pts[LEFT_SHOULDER], pts[LEFT_ELBOW], pts[LEFT_WRIST]
)

# 右手肘角度：肩膀-手肘-手腕
features["angle_right_elbow"] = calculate_angle(
    pts[RIGHT_SHOULDER], pts[RIGHT_ELBOW], pts[RIGHT_WRIST]
)

# 左肩膀角度：手肘-肩膀-髖部
features["angle_left_shoulder"] = calculate_angle(
    pts[LEFT_ELBOW], pts[LEFT_SHOULDER], pts[LEFT_HIP]
)

# 右肩膀角度：手肘-肩膀-髖部
features["angle_right_shoulder"] = calculate_angle(
    pts[RIGHT_ELBOW], pts[RIGHT_SHOULDER], pts[RIGHT_HIP]
```

```
)

# 左髖部角度：肩膀-髖部-膝蓋
features["angle_left_hip"] = calculate_angle(
    pts[LEFT_SHOULDER], pts[LEFT_HIP], pts[LEFT_KNEE]
)

# 右髖部角度：肩膀-髖部-膝蓋
features["angle_right_hip"] = calculate_angle(
    pts[RIGHT_SHOULDER], pts[RIGHT_HIP], pts[RIGHT_KNEE]
)

# 左膝蓋角度：髖部-膝蓋-腳踝
features["angle_left_knee"] = calculate_angle(
    pts[LEFT_HIP], pts[LEFT_KNEE], pts[LEFT_ANKLE]
)

# 右膝蓋角度：髖部-膝蓋-腳踝
features["angle_right_knee"] = calculate_angle(
    pts[RIGHT_HIP], pts[RIGHT_KNEE], pts[RIGHT_ANKLE]
)

# ===== 3. geometry / body ratios =====

# 先取出常用 2D 點，之後計算距離和比例會比較方便
left_shoulder = pts[LEFT_SHOULDER]
right_shoulder = pts[RIGHT_SHOULDER]
left_hip = pts[LEFT_HIP]
right_hip = pts[RIGHT_HIP]
left_elbow = pts[LEFT_ELBOW]
right_elbow = pts[RIGHT_ELBOW]
left_wrist = pts[LEFT_WRIST]
right_wrist = pts[RIGHT_WRIST]
left_knee = pts[LEFT_KNEE]
right_knee = pts[RIGHT_KNEE]
left_ankle = pts[LEFT_ANKLE]
right_ankle = pts[RIGHT_ANKLE]
```

```
# 計算肩寬
shoulder_width = calculate_distance(left_shoulder, right_shoulder)

# 計算髖寬
hip_width = calculate_distance(left_hip, right_hip)

# 計算 torso 長度：肩中點到髖中點的距離
torso_len = calculate_distance(
        ((left_shoulder[0] + right_shoulder[0]) / 2, (left_shoulder[1] +
right_shoulder[1]) / 2),
        ((left_hip[0] + right_hip[0]) / 2, (left_hip[1] + right_hip[1]) / 2)
)

# 計算左上臂長
left_upper_arm = calculate_distance(left_shoulder, left_elbow)

# 計算左前臂長
left_lower_arm = calculate_distance(left_elbow, left_wrist)

# 計算右上臂長
right_upper_arm = calculate_distance(right_shoulder, right_elbow)

# 計算右前臂長
right_lower_arm = calculate_distance(right_elbow, right_wrist)

# 計算左大腿長
left_thigh = calculate_distance(left_hip, left_knee)

# 計算左小腿長
left_calf = calculate_distance(left_knee, left_ankle)

# 計算右大腿長
right_thigh = calculate_distance(right_hip, right_knee)

# 計算右小腿長
right_calf = calculate_distance(right_knee, right_ankle)

# 很小的常數，避免分母為 0
```

```
    eps = 1e-6

    # 存肩寬
    features["shoulder_width"] = shoulder_width

    # 存髖寬
    features["hip_width"] = hip_width

    # 存 torso 長
    features["torso_len"] = torso_len

    # 存肩髖比例
    features["shoulder_hip_ratio"] = shoulder_width / (hip_width + eps)

    # 存左手臂比例：上臂 / 前臂
    features["left_arm_ratio"] = left_upper_arm / (left_lower_arm + eps)

    # 存右手臂比例：上臂 / 前臂
    features["right_arm_ratio"] = right_upper_arm / (right_lower_arm + eps)

    # 存左腿比例：大腿 / 小腿
    features["left_leg_ratio"] = left_thigh / (left_calf + eps)

    # 存右腿比例：大腿 / 小腿
    features["right_leg_ratio"] = right_thigh / (right_calf + eps)

    # 回傳該圖片的所有特徵
    return features


# 主程式進入點
def main():
    # 印出目前設定的資料集資料夾
    print("DATASET_DIR =", DATASET_DIR)

    # 若資料集資料夾不存在，印錯誤並結束
    if not os.path.exists(DATASET_DIR):
        print(f"[Error] Dataset folder not found: {DATASET_DIR}")
```

```
    return

# 確保 MediaPipe 模型存在，若沒有就下載
ensure_model_exists()

# rows 用來收集所有成功圖片的特徵資料
rows = []

# failed_images 用來記錄偵測失敗的圖片路徑
failed_images = []

# 建立 PoseLandmarker，並用 with 確保使用完自動釋放資源
with create_pose_landmarker() as landmarker:

    # 逐一處理每個類別
    for label in CLASSES:

        # 取得該類別資料夾路徑
        class_dir = os.path.join(DATASET_DIR, label)

        # 若該類別資料夾不存在，印警告並跳過
        if not os.path.exists(class_dir):
            print(f"[Warning] Class folder not found: {class_dir}")
            continue

        # 逐一讀取該資料夾中的檔案名稱
        for fname in os.listdir(class_dir):

            # 若不是合法圖片副檔名，直接跳過
            if not fname.lower().endswith(IMG_EXTENSIONS):
                continue

            # 組合出完整圖片路徑
            img_path = os.path.join(class_dir, fname)

            # 從該圖片抽取姿態特徵
            feat = extract_pose_features(img_path, landmarker)
```

```
            # 若抽特徵失敗，記錄到 failed_images，並跳過
            if feat is None:
                failed_images.append(img_path)
                continue

            # 建立這張圖的基本資訊
            row = {
                "filename": fname,     # 檔名
                "filepath": img_path,   # 完整路徑
                "label": label          # 類別標籤
            }

            # 把抽出的特徵加入 row
            row.update(feat)

            # 把該筆資料加入 rows
            rows.append(row)

# 將所有成功抽出的資料轉成 DataFrame
df = pd.DataFrame(rows)

# 匯出成 CSV，不寫入索引欄
df.to_csv(OUTPUT_CSV, index=False)

# 印出輸出檔案位置
print(f"Saved features to: {OUTPUT_CSV}")

# 印出成功圖片數量
print(f"Valid images: {len(df)}")

# 印出失敗圖片數量
print(f"Failed images: {len(failed_images)}")

# 如果有失敗圖片，印出前 20 張供檢查
if failed_images:
    print("\nExamples of failed images:")
    for p in failed_images[:20]:
        print(p)
```

```
# 若這支檔案是直接執行，而不是被 import，則執行 main()
if __name__ == "__main__":
    main()
```

(2.) train_valid_test_pose_ml.py 程式用途:

讀取 pose_features.csv，把文字標籤轉成數字，將資料切成 train / validation / test，在 train 上做 5-fold cross-validation，比較 SVM 和 RandomForest，在 validation 和 test 上輸出：

accuracy

macro-F1

confusion matrix

classification report

最後存成 train_valid_test_ml_summary.csv

```
# 匯入作業系統相關模組，用來處理檔案路徑
import os

# 匯入 NumPy，用來做數值運算、四捨五入、平均值等
import numpy as np

# 匯入 Pandas，用來讀 CSV、建立表格、輸出 summary
import pandas as pd


# 從 sklearn.model_selection 匯入資料切分與交叉驗證工具
from sklearn.model_selection import (
    # 將資料切成 train / test 或 train / valid / test
    train_test_split,

    # 建立分層 K-fold 交叉驗證器
    StratifiedKFold,

    # 執行交叉驗證並回傳多種評估結果
    cross_validate
)

# 匯入 Pipeline，用來把前處理和模型串成同一個流程
from sklearn.pipeline import Pipeline
```

```
# 匯入缺失值填補工具，這裡用平均值補缺值
from sklearn.impute import SimpleImputer

# 匯入特徵標準化工具與標籤編碼工具
from sklearn.preprocessing import StandardScaler, LabelEncoder

# 匯入 SVM 分類器
from sklearn.svm import SVC

# 匯入隨機森林分類器
from sklearn.ensemble import RandomForestClassifier

# 匯入各種分類評估指標
from sklearn.metrics import (
    # 混淆矩陣
    confusion_matrix,

    # classification report（precision / recall / f1-score）
    classification_report,

    # accuracy
    accuracy_score,

    # F1-score
    f1_score
)


# 取得目前這支 Python 檔案所在資料夾的絕對路徑
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

# 設定特徵 CSV 的路徑，預設和程式在同一個資料夾
CSV_PATH = os.path.join(BASE_DIR, "pose_features.csv")

# 設定亂數種子，確保每次切資料、訓練結果可重現
RANDOM_STATE = 42
```

```
# 最終 test set 比例 = 20%
TEST_SIZE = 0.2

# 從剩下 80% 中再切出 validation 的比例 = 12.5%
# 因為 0.8 * 0.125 = 0.1，所以最後 valid 會是 10%
VALID_SIZE_FROM_REST = 0.125

# 設定交叉驗證的 fold 數
N_SPLITS = 5


# 定義函式：讀取資料
def load_data(csv_path):
    # 如果 CSV 檔案不存在，就丟出錯誤
    if not os.path.exists(csv_path):
        raise FileNotFoundError(f"CSV file not found: {csv_path}")

    # 用 Pandas 讀取 CSV 檔
    df = pd.read_csv(csv_path)

    # 如果資料裡沒有 label 欄位，就丟出錯誤
    if "label" not in df.columns:
        raise ValueError("CSV must contain a 'label' column.")

    # 這些欄位不是模型特徵，所以之後要排除
    drop_cols = ["filename", "filepath", "label"]

    # 從所有欄位中挑出真正用來訓練的特徵欄位
    feature_cols = [c for c in df.columns if c not in drop_cols]

    # 如果一個特徵欄位都沒有，代表資料格式不對
    if len(feature_cols) == 0:
        raise ValueError("No feature columns found.")

    # 取出特徵資料 X
    X = df[feature_cols].copy()
```

```
    # 取出標籤資料 y（這時還是文字標籤）
    y = df["label"].copy()

    # 回傳：
    # df = 原始整張表
    # X = 特徵
    # y = 標籤
    # feature_cols = 特徵欄位名稱
    return df, X, y, feature_cols


# 定義函式：建立模型
def build_models():
    # 用 dict 存不同模型，key 是模型名稱，value 是模型 pipeline
    models = {
        "SVM": Pipeline([
            # 第一步：用平均值填補缺失值
            ("imputer", SimpleImputer(strategy="mean")),

            # 第二步：做特徵標準化
            ("scaler", StandardScaler()),

            # 第三步：使用 RBF kernel 的 SVM 分類器
            ("clf", SVC(kernel="rbf", C=1.0, gamma="scale"))
        ]),

        "RandomForest": Pipeline([
            # 第一步：用平均值填補缺失值
            ("imputer", SimpleImputer(strategy="mean")),

            # 第二步：隨機森林分類器
            ("clf", RandomForestClassifier(
                # 森林中樹的數量
                n_estimators=300,

                # 固定亂數種子
                random_state=RANDOM_STATE,
```

```
                # 使用所有 CPU 核心加速
                n_jobs=-1
            ))
        ])
    }

    # 回傳模型字典
    return models


# 定義函式：在 training set 上做交叉驗證
def evaluate_cv(name, model, X_train, y_train, cv):
    # 設定交叉驗證要計算的指標
    scoring = {
        # accuracy
        "accuracy": "accuracy",

        # macro-F1
        "f1_macro": "f1_macro"
    }

    # 執行交叉驗證
    results = cross_validate(
        # 要評估的模型
        model,

        # 訓練特徵
        X_train,

        # 訓練標籤
        y_train,

        # 交叉驗證器（這裡是 StratifiedKFold）
        cv=cv,

        # 要計算的指標
        scoring=scoring,
```

```
        # 不需要回傳 training score
        return_train_score=False,

        # 使用所有 CPU 核心平行計算
        n_jobs=-1
)

# 印出分隔線
print("=" * 70)

# 印出模型名稱與說明
print(f"{name} - 5-fold CV on training set")

# 印出分隔線
print("-" * 70)

# 印出每一 fold 的 accuracy
print("Fold accuracy :", np.round(results["test_accuracy"], 4))

# 印出 accuracy 平均值
print("Mean accuracy :", round(results["test_accuracy"].mean(), 4))

# 印出 accuracy 標準差
print("Std accuracy    :", round(results["test_accuracy"].std(), 4))

# 印出每一 fold 的 macro-F1
print("Fold macro-F1 :", np.round(results["test_f1_macro"], 4))

# 印出 macro-F1 平均值
print("Mean macro-F1 :", round(results["test_f1_macro"].mean(), 4))

# 印出 macro-F1 標準差
print("Std macro-F1    :", round(results["test_f1_macro"].std(), 4))

# 把交叉驗證的重點結果整理成 dict 回傳
return {
        "cv_mean_accuracy": results["test_accuracy"].mean(),
        "cv_std_accuracy": results["test_accuracy"].std(),
```

```
        "cv_mean_f1_macro": results["test_f1_macro"].mean(),
        "cv_std_f1_macro": results["test_f1_macro"].std()
    }


# 定義函式：評估某個資料切分（validation 或 test）
def evaluate_split(name, model, X_part, y_part, split_name, label_names):
    # 用已訓練好的模型進行預測
    y_pred = model.predict(X_part)

    # 計算 accuracy
    acc = accuracy_score(y_part, y_pred)

    # 計算 macro-F1
    f1_macro = f1_score(y_part, y_pred, average="macro")

    # 計算 confusion matrix
    cm = confusion_matrix(y_part, y_pred)

    # 產生 classification report
    report = classification_report(y_part, y_pred, target_names=label_names,
digits=4)

    # 印出空行加分隔線
    print("\n" + "=" * 70)

    # 印出模型名稱與當前資料集名稱
    print(f"{name} - Evaluation on {split_name} set")

    # 印出分隔線
    print("-" * 70)

    # 印出 accuracy
    print(f"{split_name} accuracy : {acc:.4f}")

    # 印出 macro-F1
    print(f"{split_name} macro-F1 : {f1_macro:.4f}")
```

```
    # 印出混淆矩陣標題
    print("\nConfusion Matrix:")

    # 把 confusion matrix 轉成 DataFrame，列名和欄名都用類別名稱
    cm_df = pd.DataFrame(cm, index=label_names, columns=label_names)

    # 印出 confusion matrix 表格
    print(cm_df)

    # 印出 classification report 標題
    print("\nClassification Report:")

    # 印出分類報告
    print(report)

    # 把評估結果整理成 dict 回傳
    return {
        # 例如 validation_accuracy 或 test_accuracy
        f"{split_name.lower()}_accuracy": acc,

        # 例如 validation_f1_macro 或 test_f1_macro
        f"{split_name.lower()}_f1_macro": f1_macro,

        # 例如 validation_confusion_matrix 或 test_confusion_matrix
        f"{split_name.lower()}_confusion_matrix": cm
    }


# 主程式
def main():
    # 讀取 CSV，拿到原始表、特徵、文字標籤、特徵欄位名稱
    df, X, y_text, feature_cols = load_data(CSV_PATH)

    # 印出目前讀到的 CSV 路徑
    print(f"Loaded CSV: {CSV_PATH}")

    # 印出總樣本數
    print(f"Total samples : {len(df)}")
```

```
# 印出特徵欄位數量
print(f"Feature count : {len(feature_cols)}")

# 印出類別分布標題
print("Class distribution:")

# 印出每一類有幾筆
print(y_text.value_counts())

# ===== label encode =====

# 建立 LabelEncoder，把文字標籤轉成數字標籤
le = LabelEncoder()

# 將文字類別（如 tree, plank）轉成數字（如 0,1,2...）
y = le.fit_transform(y_text)

# 取得數字標籤對應的原始類別名稱
label_names = list(le.classes_)

# ===== Step 1: 切出最終 test set =====
X_train_valid, X_test, y_train_valid, y_test = train_test_split(
    # 特徵
    X, y,

    # test set 比例 20%
    test_size=TEST_SIZE,

    # 分層抽樣，確保各類比例大致相同
    stratify=y,

    # 固定亂數種子
    random_state=RANDOM_STATE
)

# ===== Step 2: 從剩下 80% 中再切出 validation =====
X_train, X_valid, y_train, y_valid = train_test_split(
```

```
    # 剩下的 train+valid 特徵
    X_train_valid, y_train_valid,

    # 從剩餘資料中切出 12.5%，也就是總體的 10%
    test_size=VALID_SIZE_FROM_REST,

    # 仍然用分層抽樣
    stratify=y_train_valid,

    # 固定亂數種子
    random_state=RANDOM_STATE
)

# 印出資料切分結果標題
print("\nTrain / Validation / Test split")

# 印出 train 數量
print(f"Train size          : {len(X_train)}")

# 印出 validation 數量
print(f"Validation size : {len(X_valid)}")

# 印出 test 數量
print(f"Test size           : {len(X_test)}")

# 建立 5-fold 分層交叉驗證器
cv = StratifiedKFold(
    # 分成 5 份
    n_splits=N_SPLITS,

    # 每次先打亂再分
    shuffle=True,

    # 固定亂數種子
    random_state=RANDOM_STATE
)

# 建立模型字典
```

```
models = build_models()

# 用來收集每個模型最終 summary 的資料
summary_rows = []

# 逐一處理每個模型
for name, model in models.items():
    # 印出空行與分隔線
    print("\n" + "#" * 70)

    # 印出目前正在訓練哪個模型
    print(f"Training model: {name}")

    # 印出分隔線
    print("#" * 70)

    # ===== 1) 在 train set 上做交叉驗證 =====
    cv_result = evaluate_cv(name, model, X_train, y_train, cv)

    # ===== 2) 用完整 train set 訓練模型 =====
    model.fit(X_train, y_train)

    # ===== 3) 在 validation set 上評估 =====
    valid_result = evaluate_split(name, model, X_valid, y_valid, "Validation",
label_names)

    # ===== 4) 在 test set 上評估 =====
    test_result = evaluate_split(name, model, X_test, y_test, "Test",
label_names)

    # 把這個模型的所有結果整理成一列
    summary_rows.append({
        # 模型名稱
        "model": name,

        # train 樣本數
        "train_size": len(X_train),
```

```
        # validation 樣本數
        "valid_size": len(X_valid),

        # test 樣本數
        "test_size": len(X_test),

        # CV accuracy 平均
        "cv_mean_accuracy": cv_result["cv_mean_accuracy"],

        # CV accuracy 標準差
        "cv_std_accuracy": cv_result["cv_std_accuracy"],

        # CV macro-F1 平均
        "cv_mean_f1_macro": cv_result["cv_mean_f1_macro"],

        # CV macro-F1 標準差
        "cv_std_f1_macro": cv_result["cv_std_f1_macro"],

        # validation accuracy
        "valid_accuracy": valid_result["validation_accuracy"],

        # validation macro-F1
        "valid_f1_macro": valid_result["validation_f1_macro"],

        # test accuracy
        "test_accuracy": test_result["test_accuracy"],

        # test macro-F1
        "test_f1_macro": test_result["test_f1_macro"]
    })

# 把所有模型的結果轉成 DataFrame
summary_df = pd.DataFrame(summary_rows)

# 設定 summary 輸出檔案路徑
summary_path = os.path.join(BASE_DIR, "train_valid_test_ml_summary.csv")

# 將 summary 存成 CSV
```

```
    summary_df.to_csv(summary_path, index=False)

    # 印出分隔線
    print("\n" + "=" * 70)

    # 印出 Summary 標題
    print("Summary")

    # 印出分隔線
    print("=" * 70)

    # 依照 validation accuracy 由高到低排序後印出結果表
    print(summary_df.sort_values(by="valid_accuracy",
ascending=False).to_string(index=False))

    # 印出 summary 儲存路徑
    print(f"\nSaved summary to: {summary_path}")


# 如果這支檔案是直接執行，就呼叫 main()
if __name__ == "__main__":
    main()
```

2. 參考

(1.) https://www.google.com/search?q=yoga+pose&sca_esv=e18616a4d636eb23&hl=zh-TW&sxsrf=ANbL-n7Ex9Tk74FDMrs57GVqHm2EZzIAkw%3A1775210786749&ei=IpHPafKrLc7q1e8Pj-i52A8&biw=1280&bih=631&ved=0ahUKEwjyxOHCt9GTAxVOdfUHHQ90DvsQ4dUDCBE&uact=5&oq=yoga+pose&gs_lp=Egxnd3Mtd2l6LXNlcnAiCXlvZ2EgcG9zZUgAUABYAHAAeACQAQCYAQCgAQCqAQC4AQPIAQCYAgCgAgCYAwCSBwCgBwCyBwC4BwDCBwDIBwCACAE&sclient=gws-wiz-serp

(2.) https://www.kaggle.com/

(3.) MediaPipe / BlazePose Documentation

(4.)scikit-learn Documentation

(5.)Random Forest Documentation

(6.)SVM Documentation

(7.)OpenCV Documentation
(8.) NumPy Documentation
(9)Pandas Documentation